Linked List Palindrome

AlgoExpert · Very Hard · Linked Lists

The problem

- Do the values read the same both ways?
- Reverse the second half, compare, restore.

Intuition

- Reverse the second half, then compare it with the first half.

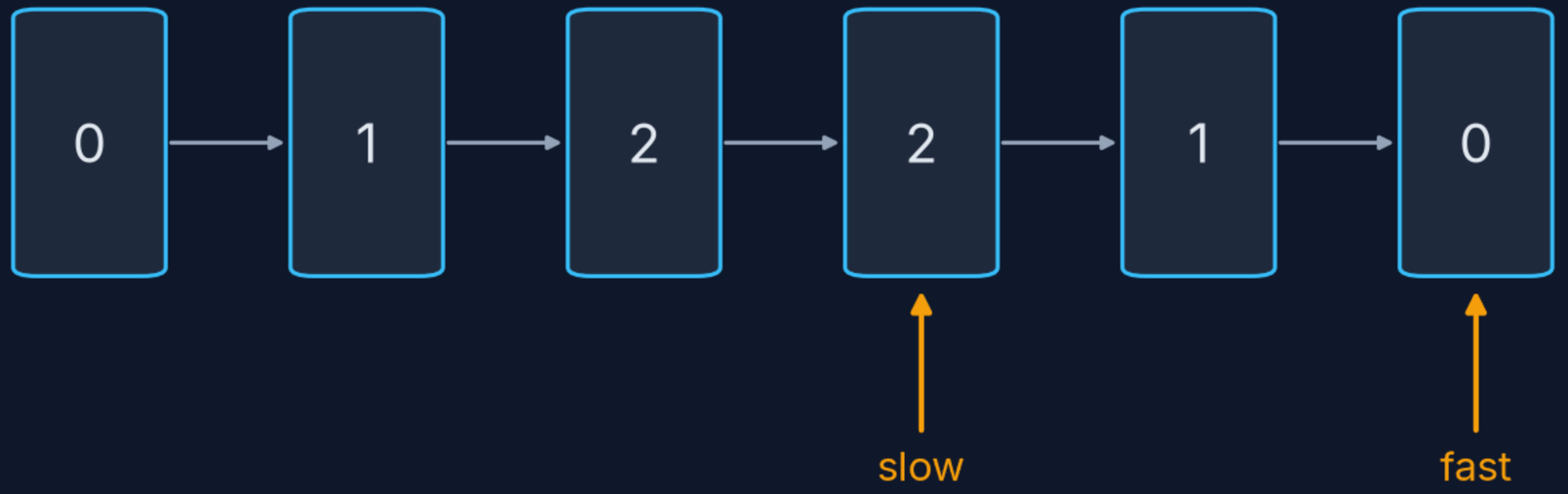
Brute force vs optimal

- Copy into array, two pointers → O(n) space
- Recursion → O(n) stack
- Reverse second half in place → O(1) space

Approach

- 1. Find the middle
- 2. Reverse the second half
- 3. Compare node by node
- 4. Restore the list

Worked example



Find the middle with slow/fast

Worked example

first half



second half reversed



Reverse the second half

Worked example





Compare node by node → true

Java solution

```java
public class LinkedListPalindrome {
    public static boolean linkedListPalindrome(LinkedListNode head) {
        LinkedListNode slow = head, fast = head;
        while (fast != null && fast.next != null) { slow = slow.next; fast = fast.next.next; }
        LinkedListNode rev = reverse(slow), a = head, b = rev;
        boolean ok = true;
        while (b != null) { if (a.value != b.value) { ok = false; break; } a = a.next; b = b.next; }
        reverse(rev);
        return ok;
    }

    private static LinkedListNode reverse(LinkedListNode h) {
        LinkedListNode p = null;
        while (h != null) { LinkedListNode n = h.next; h.next = p; p = h; h = n; }
        return p;
    }
}
```

Complexity

- Time: O(n)
- Space: O(1)

Edge cases & pitfalls

- Odd length: middle node ignored
- Restore the list afterwards
- Single node → true

Interview tips & follow-ups

- Mention the recursive approach

Recap

- Reverse the second half, then compare it with the first half.
- Complexity: O(n)